

作业 HW4* 实验报告

姓名：汪嘉晨 学号：2353900 日期：2024 年 12 月 3 日

1. 涉及数据结构和相关背景

图状结构 (Graph) 是计算机科学中的一种重要数据结构，用于表示对象及其相互关系。图由一组顶点 (节点) 和一组边 (连接这些顶点的线) 组成。图可以用于建模许多现实世界的问题，如社交网络、交通网络、网络路由等。

一、图的基本概念

1. 顶点 (Vertex)：图中的一个节点。
2. 边 (Edge)：连接两个顶点的线。
3. 有向图 (Directed Graph)：边有方向的图。
4. 无向图 (Undirected Graph)：边没有方向的图。
5. 加权图 (Weighted Graph)：边带有权重的图。
6. 邻接矩阵 (Adjacency Matrix)：一种表示图的方式，使用一个二维数组。
7. 邻接表 (Adjacency List)：另一种表示图的方式，使用一个数组的链表。

二、图的表示方法：

邻接矩阵：

使用一个二维数组 `adjMatrix`，如果顶点 `i` 和顶点 `j` 之间有边，则 `adjMatrix[i][j]` 为 1 (或权重)，否则为 0。适用于稠密图 (边数接近顶点数的平方)。

邻接表：

使用一个数组 `adjList`，数组的每个元素是一个链表，链表中存储与该顶点相邻的所有顶点。适用于稀疏图 (边数远小于顶点数的平方)。

三、图的遍历：

深度优先搜索 (DFS)：类似于树的前序遍历，使用栈 (递归实现时使用系统栈)。适用于寻找路径、连通分量等问题。

广度优先搜索 (BFS)：类似于树的层次遍历，使用队列。适用于寻找最短路径等问题。

2. 实验内容

2.1 图的遍历

2.1.1 问题描述

本题给定一个无向图，用 dfs 和 bfs 找出图的所有连通分量。

所有顶点用 0 到 $n-1$ 表示，搜索时总是从编号最小的顶点出发。使用邻接矩阵存储，或者邻接表 (使用邻接表时需要使用尾插法)。

2.1.2 基本要求

输入

第 1 行输入 2 个整数 n m ，分别表示顶点数和边数，空格分割

后面 m 行，每行输入边的两个顶点编号，空格分割

输出

第 1 行输出 dfs 的结果

第 2 行输出 bfs 的结果

连通子集输出格式为{v11 v12 ...}{v21 v22 ..}... 连通子集内元素之间用空格分割，子集之间无空格，{'和子集内第一个数字之间、}'和子集内最后一个元素之间、子集之间均无空格

对于 20%的数据，有 $0 < n \leq 15$;

对于 40%的数据，有 $0 < n \leq 100$;

对于 100%的数据，有 $0 < n \leq 1000$;

对于所有数据， $0.5n \leq m \leq 1.5n$ ，保证输入数据无错。但可能会有重边，不需特别处理。

下载 p107_data.cpp，编译运行以生成随机测试数据

2.1.3 数据结构设计

```
typedef struct MGraph {
    int vexs[MAX_NUM] = { 0 }; // 节点
    vector<vector<int>> arcs; // 邻接矩阵
    int vexnum, arcnum;
    MGraph() : arcs(MAX_NUM, vector<int>(MAX_NUM, 0)) {

    }
} MGraph;

class Picture {
public:
    enum SEARCH_TYPE { dfs, bfs };
    void CreateMGraph() {
        cin >> G.vexnum >> G.arcnum;
        for (int i = 0; i < G.arcnum; i++) {
            int y, x;
            cin >> y >> x;
            G.arcs[y][x] = 1;
            G.arcs[x][y] = 1;
        }
    }
    void Search(SEARCH_TYPE type) {
        for (int i = 0; i < MAX_NUM; i++)
            visited[i] = 0;
        for (int i = 0; i < G.vexnum; i++) {
            if (!visited[i]) {
                if (type == SEARCH_TYPE::dfs)
                    SearchByDfs(i);
                else
                    SearchByBfs(i);
                cout << '{';
                while (!DFS_BFS.empty()) {
```

```

        cout << DFS_BFS.front();
        DFS_BFS.pop_front();
        if (!DFS_BFS.empty())
            cout << ' ';
    }
    cout << '}' ;
}
}
cout << endl;
}
private:
    MGraph G;
    bool visited[MAX_NUM];
    deque<int> DFS_BFS;
    void SearchByBfs(int start) {
        queue<int> Q;
        Q.push(start);
        visited[start] = 1;
        while (!Q.empty()) {
            int now = Q.front();
            Q.pop();
            DFS_BFS.push_back(now);
            for (int i = 0; i < G.vexnum; i++) {
                if (G.arcs[now][i] && !visited[i]) {
                    Q.push(i);
                    visited[i] = 1;
                }
            }
        }
    }
    void SearchByDfs(int now) {
        DFS_BFS.push_back(now);
        visited[now] = 1;
        for (int i = 0; i < G.vexnum; i++) {
            if (G.arcs[now][i] && !visited[i]) {
                SearchByDfs(i);
            }
        }
    }
};

```

2.1.4 功能说明（函数、类）

1.MGraph 构造函数

初始化邻接矩阵为 MAX_NUM x MAX_NUM 的零矩阵。

2. Picture::CreateMGraph()

从标准输入读取图的顶点数和边数。

读取每条边的两个顶点，并在邻接矩阵中标记相应的边。

3. Picture::Search(SEARCH_TYPE type)

初始化访问标记数组 visited。

遍历所有顶点，如果顶点未被访问过，则根据搜索类型调用 SearchByDfs 或 SearchByBfs 进行搜索。

输出搜索结果，格式为 {v1 v2 ...}。

4. Picture::SearchByBfs(int start)

使用队列实现广度优先搜索 (BFS)。

从起始顶点开始，依次访问所有相邻未被访问的顶点，并将其加入队列。

将访问过的顶点加入 DFS_BFS 队列。

5. Picture::SearchByDfs(int now)

使用递归实现深度优先搜索 (DFS)。

从当前顶点开始，依次访问所有相邻未被访问的顶点。

将访问过的顶点加入 DFS_BFS 队列。

2.1.5 调试分析（遇到的问题 and 解决方法）

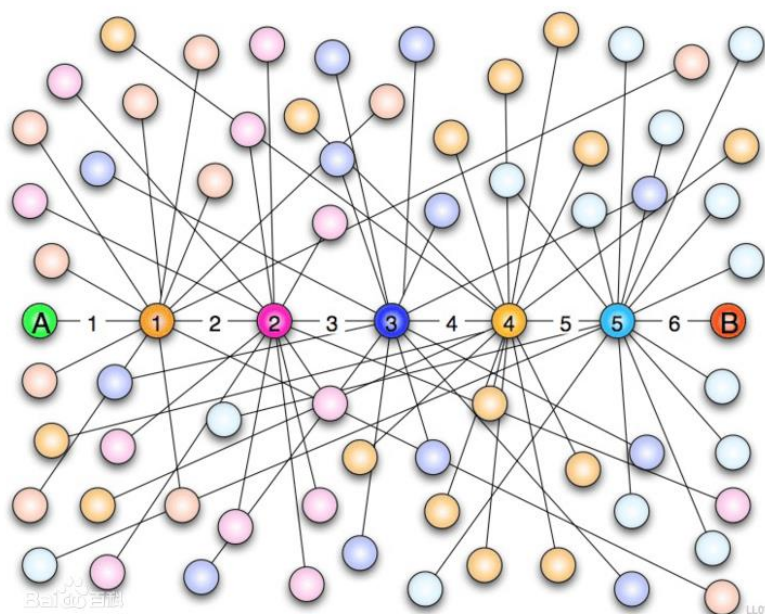
在 SearchByDfs 函数中，递归调用可能导致栈溢出。解决方法：对于大规模数据，考虑使用非递归的深度优先搜索实现。

2.1.6 总结和体会

在本题中，使用邻接矩阵存储图，并使用 BFS 和 DFS 算法进行遍历，能够高效地找到所有连通分量。

2.2 六度空间

2.2.1 问题描述



六度空间理论又称小世界理论。理论通俗地解释为：“你和世界上任何一个陌生人之间所间隔的人不会超过 6 个人，也就是说，最多通过五个人你就能够认识任何一个陌生人。”如图 1 所示。假如给你一个社交网络图，请你对每个节点计算符合“六度空间”理论的结点占结点总数的百分比。说明：由于浮点数精度不同导致结果有误差，请按 float 计算。

2.2.2 基本要求

输入

第 1 行给出两个正整数，分别表示社交网络图的结点数 N ($1 < N \leq 2000$ ，表示人数)、边数 M ($\leq 33 \times N$ ，表示社交关系数)。

随后的 M 行对应 M 条边，每行给出一对正整数，分别是该条边直接连通的两个结点的编号（节点从 1 到 N 编号）。

输出

对每个结点输出与该结点距离不超过 6 的结点数占结点总数的百分比，精确到小数点后 2 位。每个结节点输出一行，格式为“结点编号: (空格) 百分比%”。

2.2.3 数据结构设计

```
vector<int> adj[MAXN + 1]; // 邻接表
```

2.2.4 功能说明（函数、类）

```
// 广度优先搜索函数，计算从 start 节点出发，距离不超过 6 的结点数
float bfs(int start, int n) {
    vector<int> distance(n + 1, -1); // 记录每个节点的距离，初始为-1
    queue<int> q; // 队列用于BFS
    q.push(start); // 起始节点入队
    distance[start] = 0; // 起始节点距离为0
    int count = 0; // 计数器

    while (!q.empty()) {
        int node = q.front(); // 取队首节点
        q.pop(); // 队首节点出队
        if (distance[node] > 6) break; // 如果距离超过6，停止搜索
        count++; // 计数器加1
        for (int neighbor : adj[node]) { // 遍历当前节点的所有邻居
            if (distance[neighbor] == -1) { // 如果邻居节点未被访问
                distance[neighbor] = distance[node] + 1; // 更新邻居节点的距离
                q.push(neighbor); // 邻居节点入队
            }
        }
    }

    return (float)count / n * 100; // 计算百分比并返回
}

int main() {
```

```

int n, m;
cin >> n >> m; // 读取节点数和边数
for (int i = 0; i < m; ++i) {
    int u, v;
    cin >> u >> v; // 读取每条边
    adj[u].push_back(v); // 构建邻接表
    adj[v].push_back(u); // 无向图，双向添加
}

for (int i = 1; i <= n; ++i) {
    float percentage = bfs(i, n); // 计算每个节点的百分比
    cout << i << ": " << fixed << setprecision(2) << percentage <<
    "%" << endl; // 输出结果
}

return 0;
}

```

2.2.5 调试分析（遇到的问题和解决方法）

问题：浮点数计算的精度问题

解决方法：在计算百分比时，使用 `float` 类型，并使用 `setprecision` 来设置输出的精度。可以参考 C++ 标准库中的 `<iomanip>` 头文件的使用方法

2.2.6 总结和体会

1. 广度优先搜索 (BFS):

BFS 是一种遍历图的有效算法，适用于查找最短路径和计算节点之间的距离。

使用队列来实现 BFS，可以确保按层次遍历节点。

2. 邻接表的使用:

邻接表是一种高效的图表示方法，特别适用于稀疏图。

使用 `vector` 来存储每个节点的邻居节点，可以方便地进行图的遍历和操作。

2.3 村村通

2.3.1 问题描述

N 个村庄，从 1 到 N 编号，现在请你修建一些路使得任何两个村庄都彼此连通。我们称两个村庄 A 和 B 是连通的，当且仅当在 A 和 B 之间存在一条路，或者存在一个村庄 C，使得 A 和 C 之间有一条路，并且 C 和 B 是连通的。

已知在一些村庄之间已经有了一些路，您的工作是再兴建一些路，使得所有的村庄都是连通的，并且新建的路的长度是最小的。

2.3.2 基本要求

输入

第一行包含一个整数 n ($3 \leq n \leq 100$)，表示村庄数目。

接下来 n 行, 每行 n 个非负整数，表示村庄 i 和村庄 j 之间的距离。距离值在 [1, 1000] 之间。

接着是一个整数 m ，后面给出 m 行，每行包含两个整数 a, b ($1 \leq a < b$)，表示在村庄 a 和 b 之间已经修建了路。

下载编译并运行 p134.cpp 生成随机测试数据

输出

输出一行，仅有一个整数，表示为使所有的村庄连通，要新建公路的长度的最小值。

数据范围

$3 \leq n \leq 100$ 。

边权都是 $[1, 1000]$ 之间的整数。

2.3.3 数据结构设计

```
// 边结构体，包含两个端点和边的权重
struct Edge {
    int u, v, weight;
    // 重载小于运算符，用于边的排序
    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};
```

2.3.4 功能说明（函数、类）

```
// 并查集查找函数，带路径压缩
int find(vector<int>& parent, int i) {
    if (parent[i] != i) {
        parent[i] = find(parent, parent[i]);
    }
    return parent[i];
}

// 并查集合并函数，带秩优化
void unite(vector<int>& parent, vector<int>& rank, int u, int v) {
    int rootU = find(parent, u);
    int rootV = find(parent, v);
    if (rootU != rootV) {
        if (rank[rootU] > rank[rootV]) {
            parent[rootV] = rootU;
        }
        else if (rank[rootU] < rank[rootV]) {
            parent[rootU] = rootV;
        }
        else {
            parent[rootV] = rootU;
            rank[rootU]++;
        }
    }
}
```

```

}

int main() {
    int n;
    cin >> n; // 读取村庄数目
    vector<Edge> edges;
    vector<vector<int>> dist(n, vector<int>(n));

    // 读取距离矩阵
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            cin >> dist[i][j];
            if (i < j) {
                edges.push_back({ i, j, dist[i][j] });
            }
        }
    }

    int m;
    cin >> m; // 读取已经修建的道路数目
    vector<int> parent(n), rank(n, 0);
    for (int i = 0; i < n; ++i) {
        parent[i] = i;
    }

    // 处理已经修建的道路
    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        unite(parent, rank, a - 1, b - 1);
    }

    // 对所有边按权重排序
    sort(edges.begin(), edges.end());

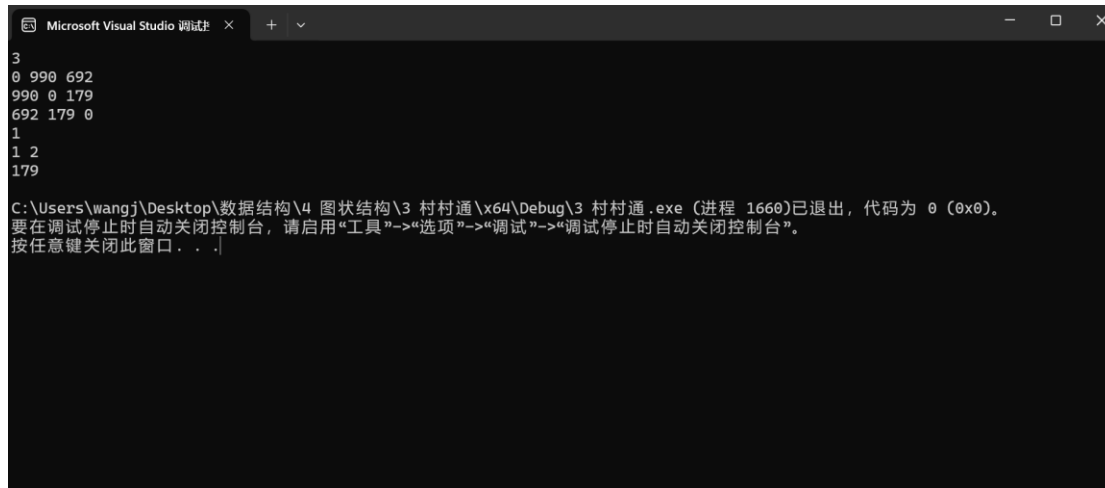
    int mst_weight = 0;
    // Kruskal 算法构建最小生成树
    for (const auto& edge : edges) {
        if (find(parent, edge.u) != find(parent, edge.v)) {
            unite(parent, rank, edge.u, edge.v);
            mst_weight += edge.weight;
        }
    }
}

```



```
// 输出最小生成树的权重
cout << mst_weight << endl;
return 0;
}
```

2.3.5 调试分析（遇到的问题 and 解决方法）



2.3.6 总结和体会

1.Kruskal 算法的应用：

Kruskal 算法是一种贪心算法，适用于边稀疏的图。它通过逐步选择权重最小的边来构建最小生成树。该算法的关键步骤包括对所有边按权重排序，以及使用并查集（Union-Find）来检测和避免环的形成。

2.并查集的使用：用于处理不相交集合并和查找操作。在 Kruskal 算法用于检测是否形成环。路径压缩和按秩合并是并查集的两种优化技术，可以显著提高查找和合并操作的效率。

2.4 给定条件下构造矩阵

2.4.1 问题描述

给你一个 正整数 k ，同时给你：

一个大小为 n 的二维整数数组 $rowConditions$ ，其中 $rowConditions[i] = [above_i, below_i]$ 和一个大小为 m 的二维整数数组 $colConditions$ ，其中 $colConditions[i] = [left_i, right_i]$ 。

两个数组里的整数都是 1 到 k 之间的数字。

你需要构造一个 $k \times k$ 的矩阵， 1 到 k 每个数字需要 恰好出现一次 。剩余的数字都是 0 。

矩阵还需要满足以下条件：

对于所有 0 到 $n - 1$ 之间的下标 i ，数字 $above_i$ 所在的 行 必须在数字 $below_i$ 所在行的上面。对于所有 0 到 $m - 1$ 之间的下标 i ，数字 $left_i$ 所在的 列 必须在数字 $right_i$ 所在列的左边。返回满足上述要求的矩阵，题目保证若矩阵存在则一定唯一；如果不存在答案，返回一个空的矩阵。

2.4.2 基本要求

输入

第一行包含 3 个整数 k 、 n 和 m

接下来 n 行，每行两个整数 abovei、belowi，描述 rowConditions 数组

接下来 m 行，每行两个整数 lefti、righti，描述 colConditions 数组

输出

如果可以构造矩阵，打印矩阵；否则输出-1

矩阵中每行元素使用空格分隔

2.4.3 数据结构设计

```
// 构建矩阵函数
vector<vector<int>> buildMatrix(int k, const vector<pair<int, int>>&
rowConditions, const vector<pair<int, int>>& colConditions) {
    vector<int> rowOrder = topologicalSort(k, rowConditions); // 行的拓扑
    排序
    vector<int> colOrder = topologicalSort(k, colConditions); // 列的拓扑
    排序

    if (rowOrder.empty() || colOrder.empty()) {
        return {}; // 如果任一排序结果为空，返回空矩阵
    }

    unordered_map<int, int> rowIndex, colIndex;
    for (int i = 0; i < k; ++i) {
        rowIndex[rowOrder[i]] = i; // 记录每个元素在行中的位置
        colIndex[colOrder[i]] = i; // 记录每个元素在列中的位置
    }

    vector<vector<int>> matrix(k, vector<int>(k, 0)); // 初始化 k x k 的矩
    阵
    for (int i = 1; i <= k; ++i) {
        matrix[rowIndex[i]][colIndex[i]] = i; // 根据行列位置填充矩阵
    }

    return matrix;
}
```

2.4.4 功能说明（函数、类）

```
// 拓扑排序函数，返回拓扑排序后的节点顺序
vector<int> topologicalSort(int k, const vector<pair<int, int>>&
conditions) {
    vector<int> inDegree(k + 1, 0); // 入度数组
    unordered_map<int, vector<int>> graph; // 邻接表表示的图
    for (const auto& condition : conditions) {
        graph[condition.first].push_back(condition.second);
        inDegree[condition.second]++;
    }
}
```

```

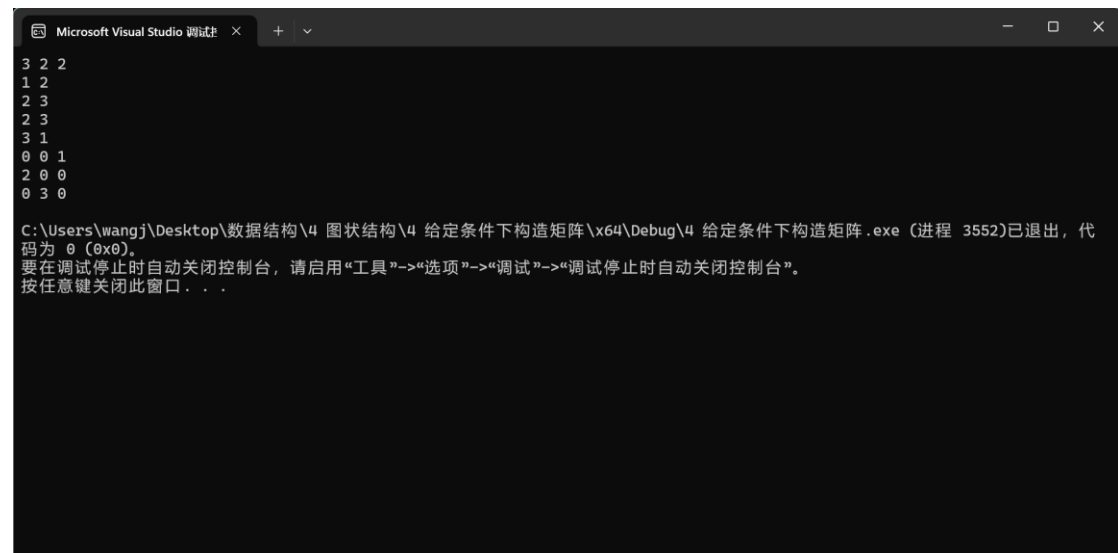
queue<int> q;
for (int i = 1; i <= k; ++i) {
    if (inDegree[i] == 0) {
        q.push(i); // 入度为0 的节点入队
    }
}

vector<int> order;
while (!q.empty()) {
    int node = q.front();
    q.pop();
    order.push_back(node);
    for (int neighbor : graph[node]) {
        if (--inDegree[neighbor] == 0) {
            q.push(neighbor); // 入度为0 的邻居节点入队
        }
    }
}

return order.size() == k ? order : vector<int>(); // 如果排序结果包含
所有节点则返回排序结果，否则返回空
}

```

2.4.5 调试分析（遇到的问题解决方法）



2.4.6 总结和体会

题目的关键在于根据行约束和列约束分别求出两个符合条件的一维数组序列，再将所得到的两个有序的一维数组序列转化为二维度矩阵（行排序条件和列排序条件在本题是正交的，因此可以转化）

2.5 必修课

2.5.1 问题描述

某校的计算机系有 n 门必修课程。学生需要修完所有必修课程才能毕业。

每门课程都需要一定的学时去完成。有些课程有前置课程，需要先修完它们才能修这些课程；而其他课程没有。不同于大多数学校，学生可以在任何时候进行选课，且同时选课的数量没有限制。

现在校方想要知道：

1. 从入学开始，每门课程最早可能完成的时间（单位：学时）；
2. 对每一门课程，若将该课程的学时增加 1，是否会延长入学到毕业的最短时间。

2.5.2 基本要求

输入

第一行，一个正整数 n ，代表课程的数量。

接下来 n 行，每行若干个整数：

- 第一个整数为 t_i ，表示修完该课程所需的学时。
- 第二个整数为 c_i ，表示该课程的前置课程数量。
- 接下来 c_i 个互不相同的整数，表示该课程的前置课程的编号。

该校保证，每名入学的学生，一定能够在有限的时间内毕业。

输出

输出共 n 行，第 i 行包含两个整数：

- 第一个整数表示编号为 i 的课程最早可能完成的时间。
- 第二个整数表示，如果将该课程的学时增加 1，入学到毕业的最短时间是否会增加。如果会增加则输出 11，否则输出 00。

每行的两个整数以一个空格隔开。

2.5.3 数据结构设计

```
struct Subject {
    int time_consuming;    // 累计所需时间
    int last_subjects, next_subjects; // 前驱科目数量和后继科目数量
    // 科目开始的最早和最晚时间
    int min_start_time; // ve
    int max_start_time; // vl
    std::vector<int> next; // 后继科目列表
    std::vector<int> last; // 前驱科目列表
    bool important;
    Subject() : last_subjects(0), next_subjects(0),
min_start_time(0), max_start_time(INT_MAX), important(0) {}
};

int size;
std::vector<Subject> subjects;
```

2.5.4 功能说明（函数、类）

```

public:
    // 构造函数，生成具有指定参数的随机图
    void f_in() {
        // 科目数量
        cin >> size;
        for (int i = 0; i < size; i++) {
            Subject subject;
            cin >> subject.time_consuming; // time
            cin >> subject.last_subjects; // 前驱个数
            for (int j = 0; j < subject.last_subjects; j++) {
                int k;
                cin >> k;
                subject.last.push_back(k - 1);
            }
            subjects.push_back(subject);
        }
        // 手动加入结束事件
        Subject subject;
        subject.time_consuming = 0;
        subject.next_subjects = 0;
        subjects.push_back(subject);
        // 填充 next 的课程
        for (int i = 0; i < size; i++) {
            for (int j : subjects[i].last) {
                subjects[j].next.push_back(i);
                subjects[j].next_subjects++;
            }
        }
        // 如果没有 next 就指向结束事件
        for (int i = 0; i < size; i++) {
            if (subjects[i].next_subjects == 0) {
                subjects[size].last_subjects++;
                subjects[size].last.push_back(i);
                subjects[i].next_subjects++;
                subjects[i].next.push_back(size);
            }
        }
        size++;
    }

    // 计算关键路径
    bool calculateKeyWay() {
        queue<int> que;
        stack<int> sta;
    }

```

```

    int count = 0;
    // 从无前驱科目的节点开始拓扑排序并计算min_start_time (ve)
    for (int i = 0; i < size; i++) {
        if (subjects[i].last_subjects == 0) {
            que.push(i);
        }
    }
    while (count < size && !que.empty()) {
        int j = que.front();
        que.pop();
        sta.push(j);
        count++;
        for (auto next : subjects[j].next) {
            if (--subjects[next].last_subjects == 0)
                que.push(next);
            subjects[next].min_start_time =
max(subjects[next].min_start_time, subjects[j].min_start_time +
subjects[j].time_consuming);
        }
    }
    if (count != size || !que.empty()) {
        return false;
    }
    // 计算max_start_time(vl)
    for (int i = 0; i < size; i++) {
        subjects[i].max_start_time = subjects[size -
1].min_start_time;
    }
    while (!sta.empty()) {
        int j = sta.top();
        sta.pop();
        for (int i : subjects[j].next) {
            subjects[j].max_start_time =
min(subjects[i].max_start_time - subjects[j].time_consuming,
subjects[j].max_start_time);
        }
    }
    // e l 关键路径
    for (int i = 0; i < size - 1; i++) {
        for (int j : subjects[i].next) {
            int e = subjects[i].min_start_time;
            int l = subjects[j].max_start_time -
subjects[i].time_consuming;
            if (e == l) {

```

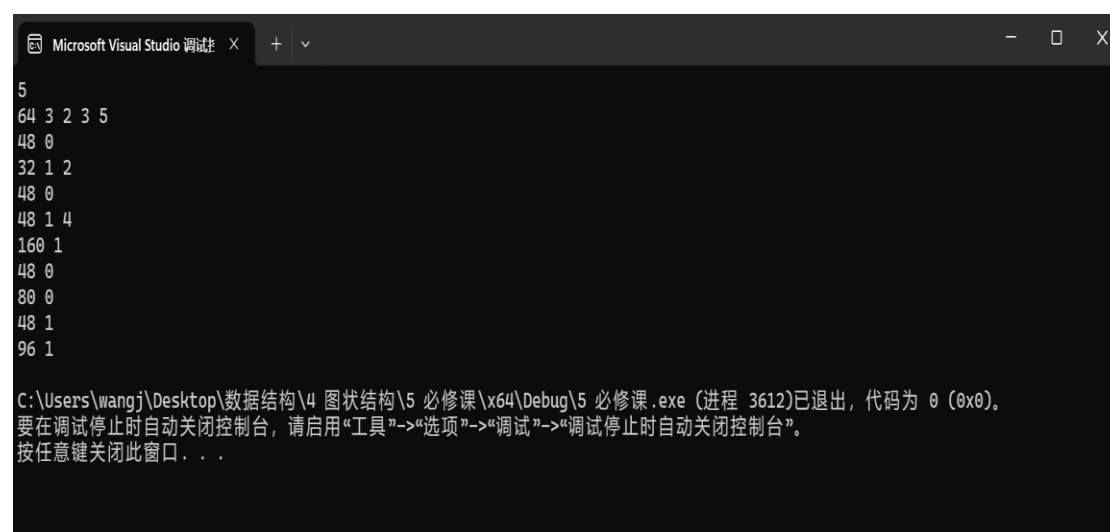
```

        subjects[i].important = 1;
        break;
    }
}
return true;
}

// 打印答案
void printAns() {
    if (!calculateKeyWay()) {
        cout << "error!无法拓扑排序!" << endl;
        return;
    }
    subjects.pop_back();
    for (Subject i : subjects) {
        cout << i.min_start_time + i.time_consuming << ' ' <<
i.important << endl;
    }
}

```

2.5.5 调试分析（遇到的问题 and 解决方法）



The screenshot shows the Microsoft Visual Studio Debug Console. The output of the program is as follows:

```

5
64 3 2 3 5
48 0
32 1 2
48 0
48 1 4
160 1
48 0
80 0
48 1
96 1

```

Below the output, an error message is displayed:

```

C:\Users\wangj\Desktop\数据结构\4 图状结构\5 必修课\x64\Debug\5 必修课.exe (进程 3612)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . .

```

2.5.6 总结和体会

1. 前置课程和后继课程的处理：确保 last 和 next 列表正确填充。
2. 拓扑排序的实现：使用队列和栈来实现拓扑排序，并计算 min_start_time 和 max_start_time。
3. 关键路径的计算：通过计算 min_start_time 和 max_start_time 来判断课程是否在关键路径上。

3.实验总结

图状结构是之前学到的链表，栈与队列，树等的集大成者：

